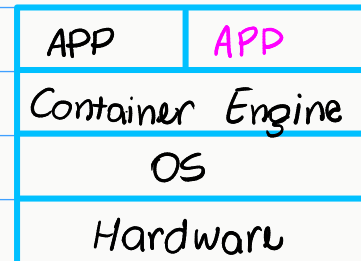


VM vs Containers

Virtual Machine

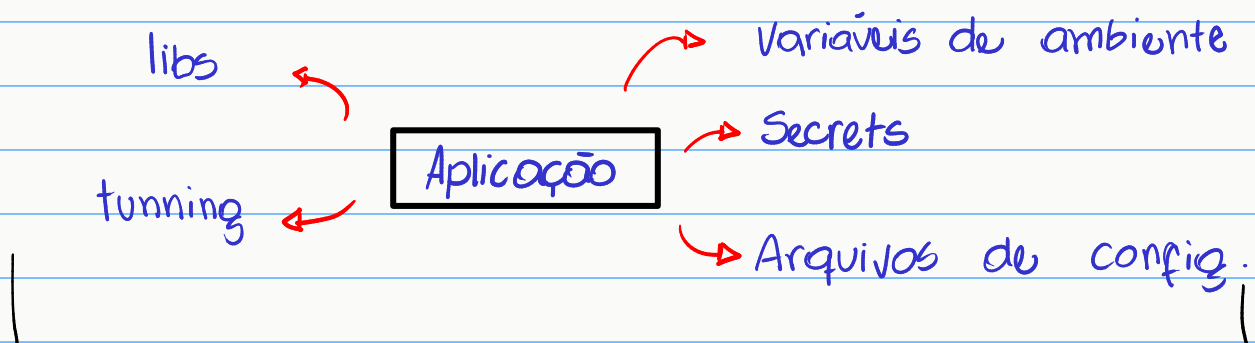


Container



- * Otimização de recursos
- * Isolamento
- * Imutabilidade
- * Deploy facilitado

Por que usar Docker?



↳ Posso colocar tudo isso em uma imagem.

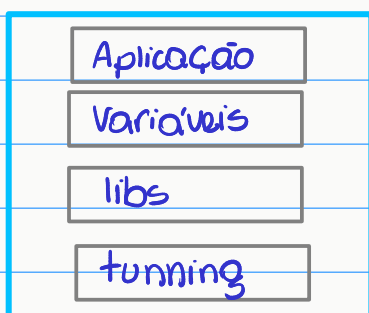


Imagem docker

→ Imutável

↳ Uma imagem é sempre read-only.

Posso deployar a imagem em diferentes lugares

Imagem docker

↳ Imagem imutável

↳ EC2
↳ EKS
↳ On-premises
↳ GCP
↳ Azure

↳ Roda em todos os lugares desde que:

Container Engine

→ Gera imagem → write

Imagem docker

→ read-only

Normas

Open Container Initiative

OCI

Runtime Specification

↳ Image Specification

→ Docker

→ Cri-o

↳ Containerd

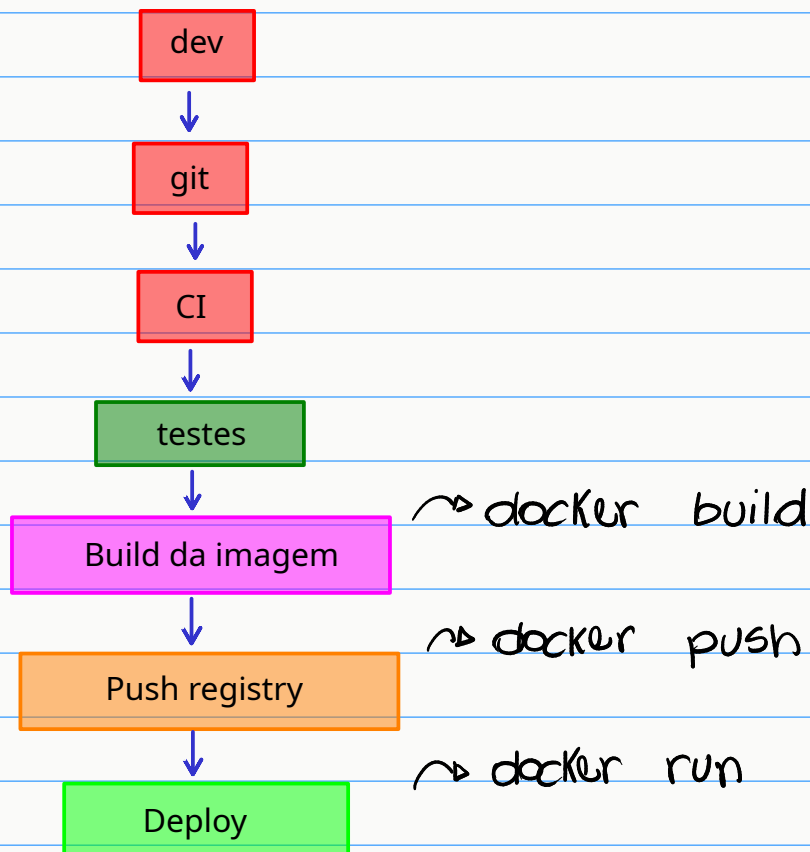
→ Docker

→ Kaniko

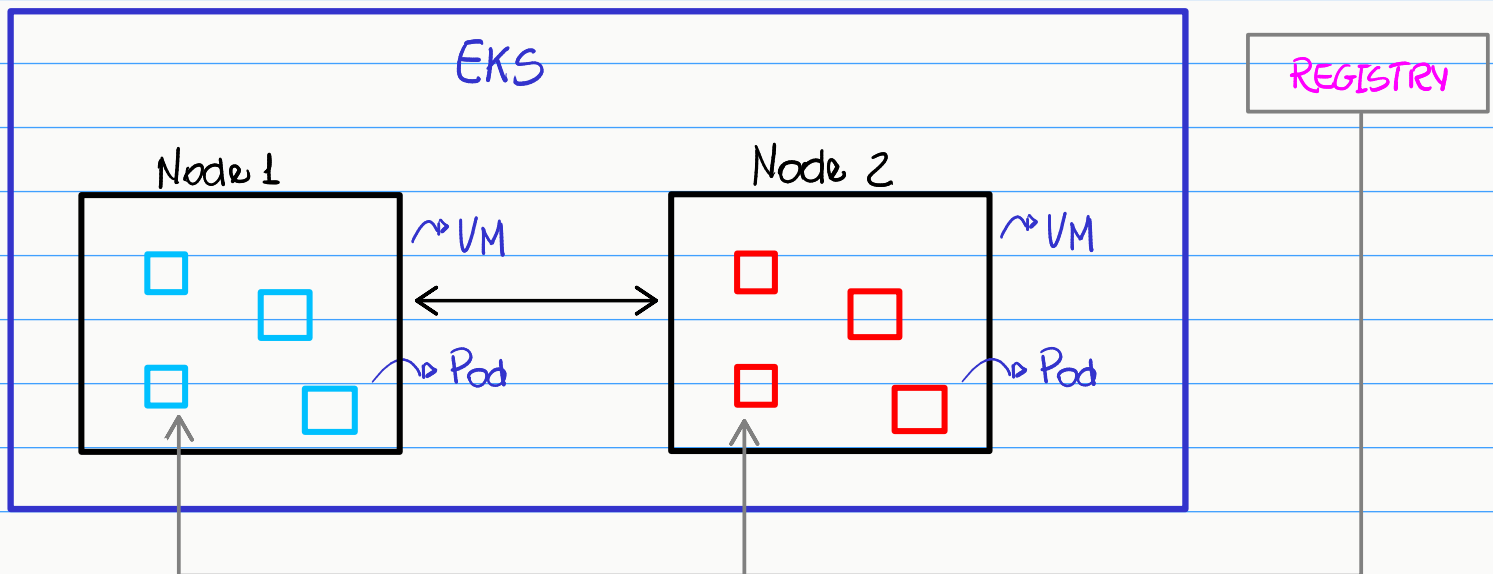
→ Buildah

Como Docker é utilizado nas empresas

↳ Continuous Integration



DEPLOY → Continuous Deploy



Princípios do Linux

chroot → É utilizado para isolamento de diretórios

↳ Consigo criar ou mudar o dir. raíz com meus próprios binários.

↳ ls

↳ ps

↳ vi

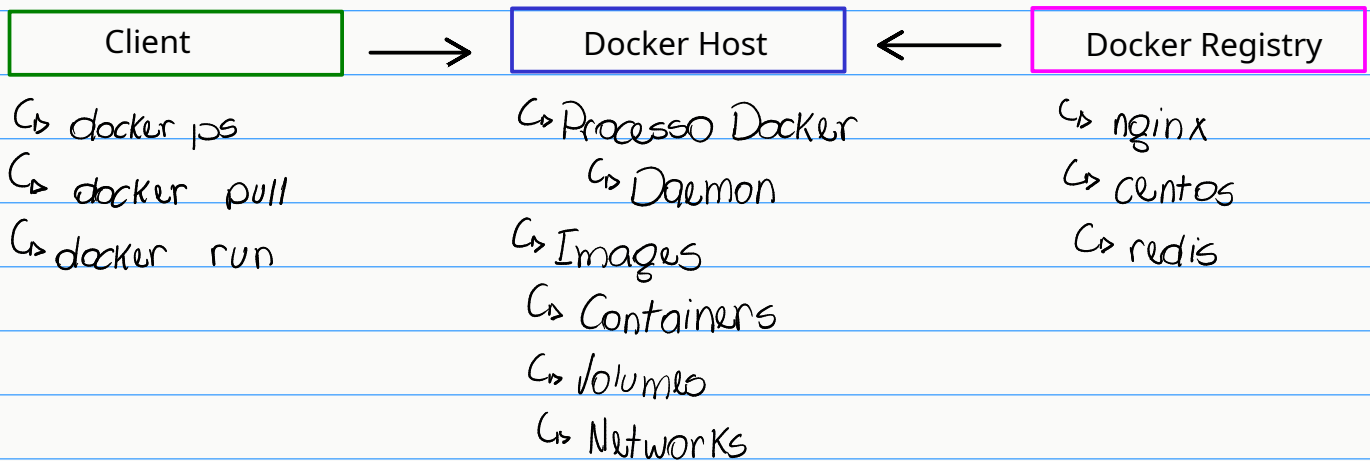
namespaces → Utilizado para isolamento de processos.

↳ Posso isolar outras coisas com comando unshare.

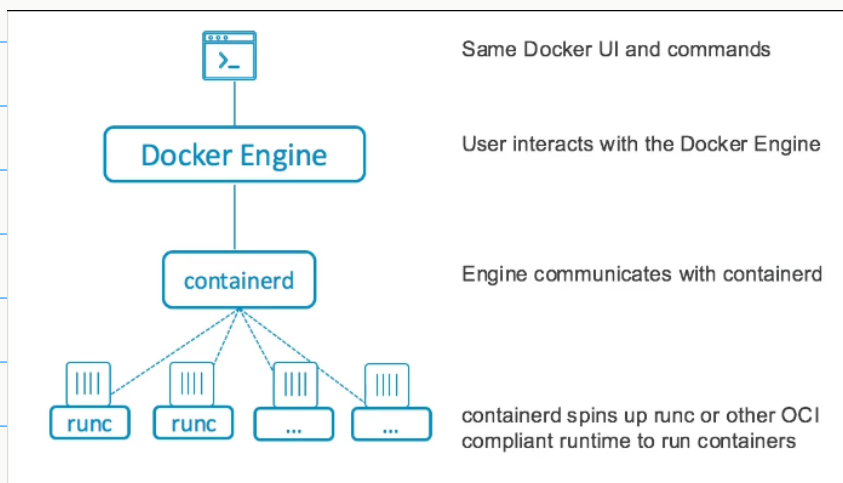
↳ nesse caso serão isolados processos.

cgroups → Utilizado para isolar recursos de hardware.
↳ Criar VM para simular consumo de CPU.

Arquitetura do Docker



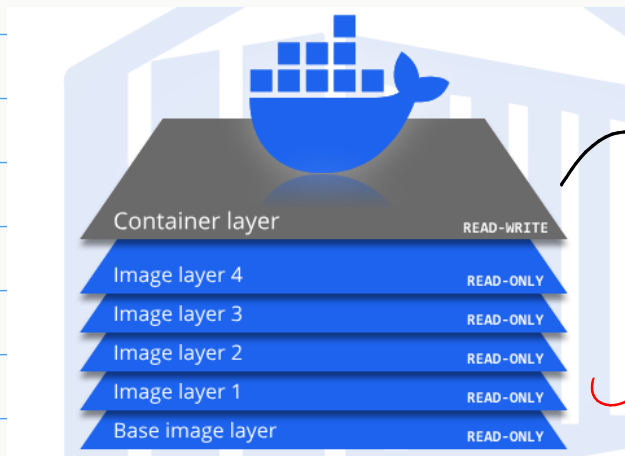
Docker Engine



→ Abstração p/ usuário.

→ Realmente gerencia o ciclo de vida dos containers.

Imagem vs Container

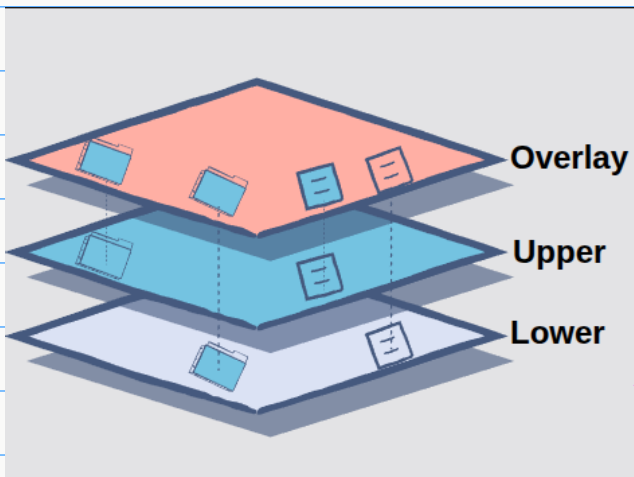


Quando um container nasce ou morre apenas essa camada é adicionada ou removida.

Imagens são imutáveis.

OverlayFS

Uma imagem é composta por várias camadas.



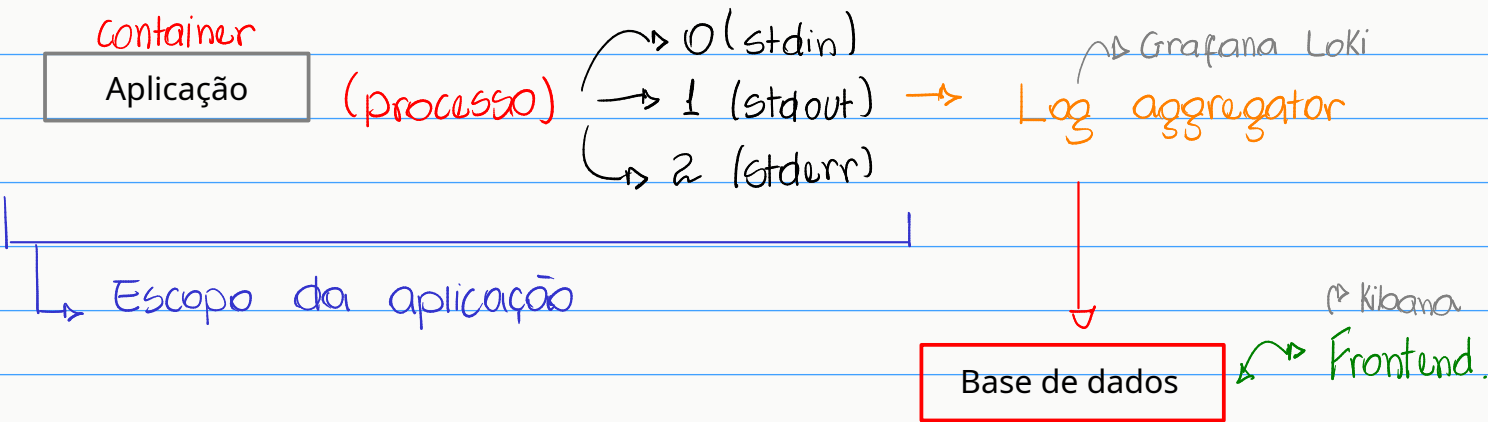
Cada camada é um diretório dentro de `/var/lib/docker/overlay`

Importante no Docker host

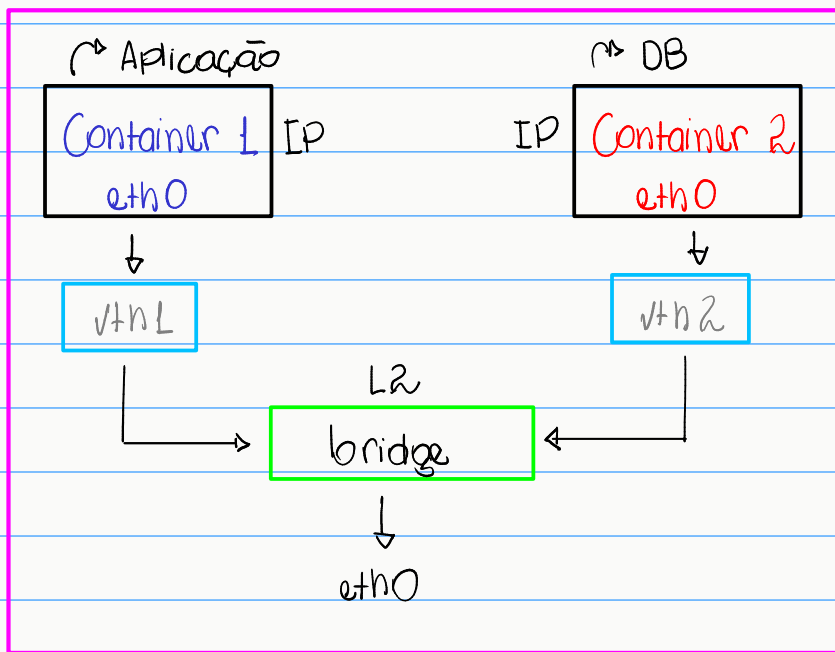
faz a junção de todas, sendo que o mais alto sempre tem preferência

Cada layer possui a referência p layer pai, e consigo ver a diferença entre eles.

Logs em containers

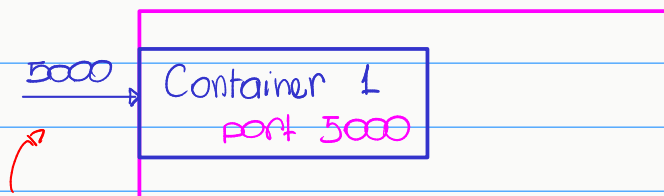


Docker Networking



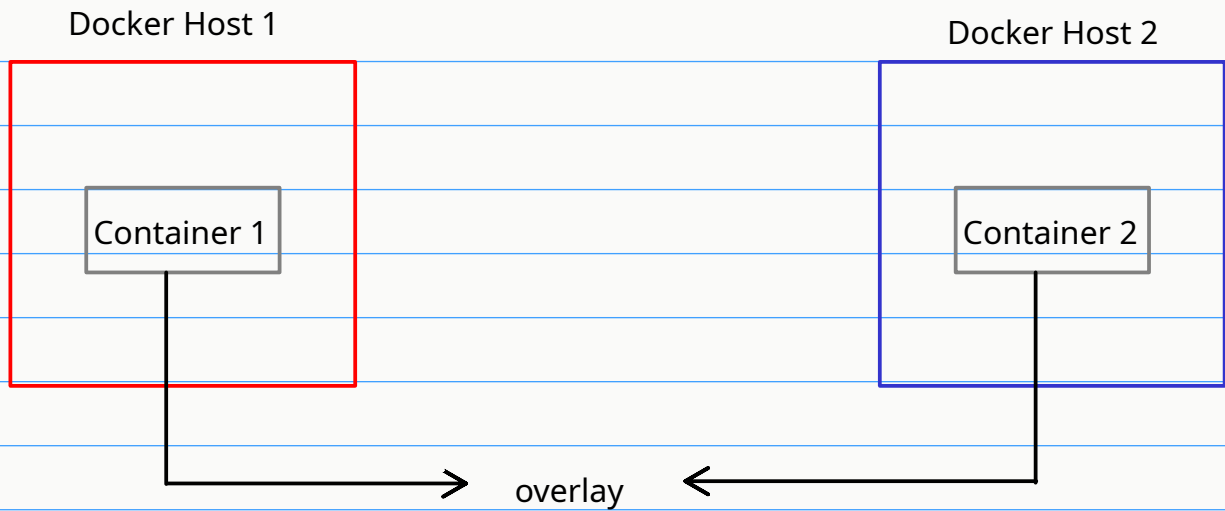
Host

Docker Host



conexão externa, não é uma boa prática.

↳ Overlay (Docker swarm)



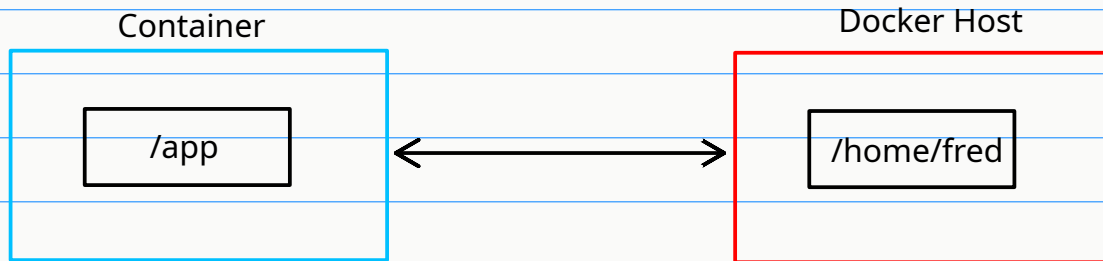
↳ forma um túnel de comunicação

Docker Storage

↳ Consigo persistir o estado de algo do container.

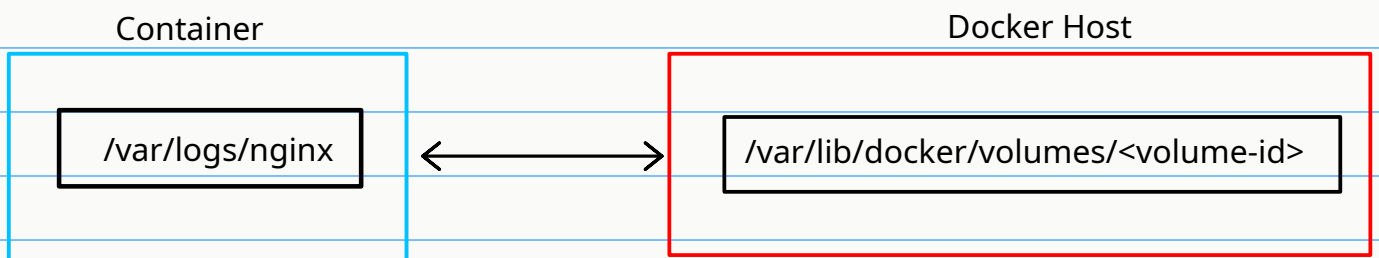
↳ Bind Mounting

Monta um diretório ou arquivo do host para dentro do container.



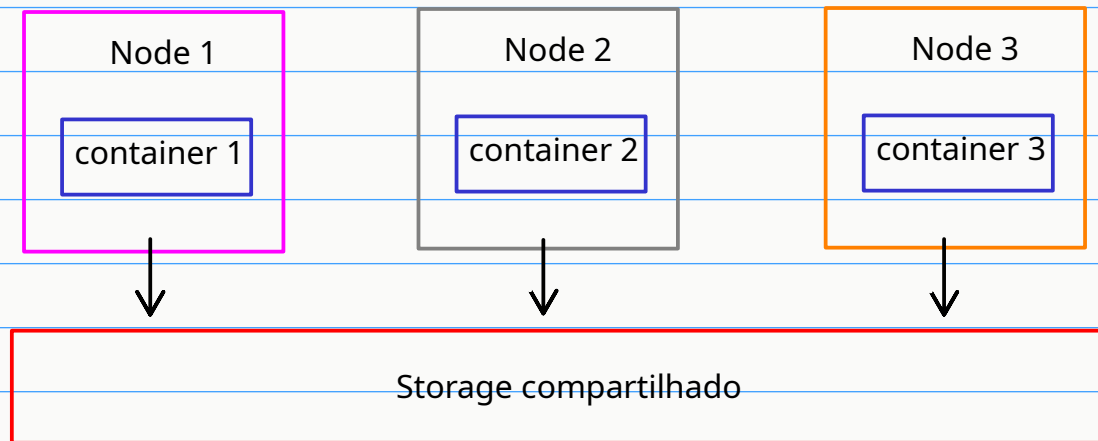
↳ Volume

↳ Diretório do host escolhido de forma aleatória.

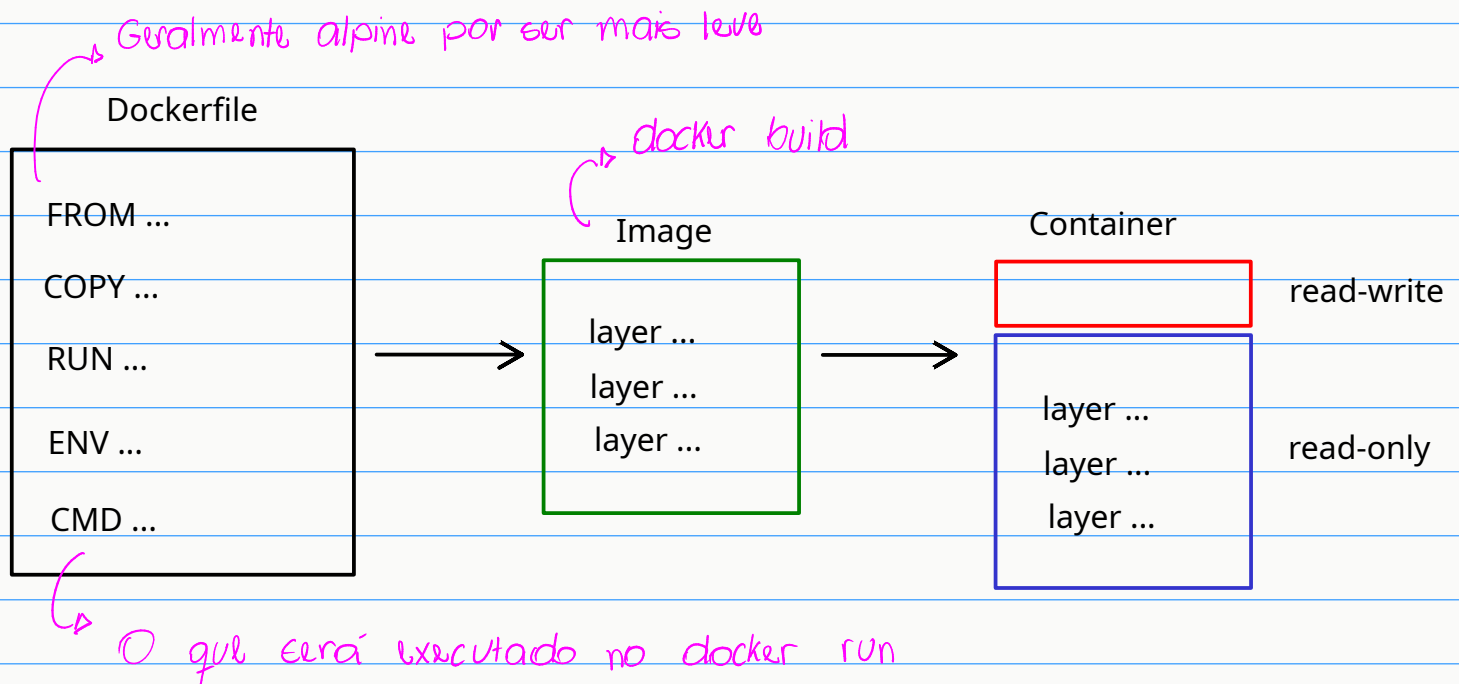


C Distributed

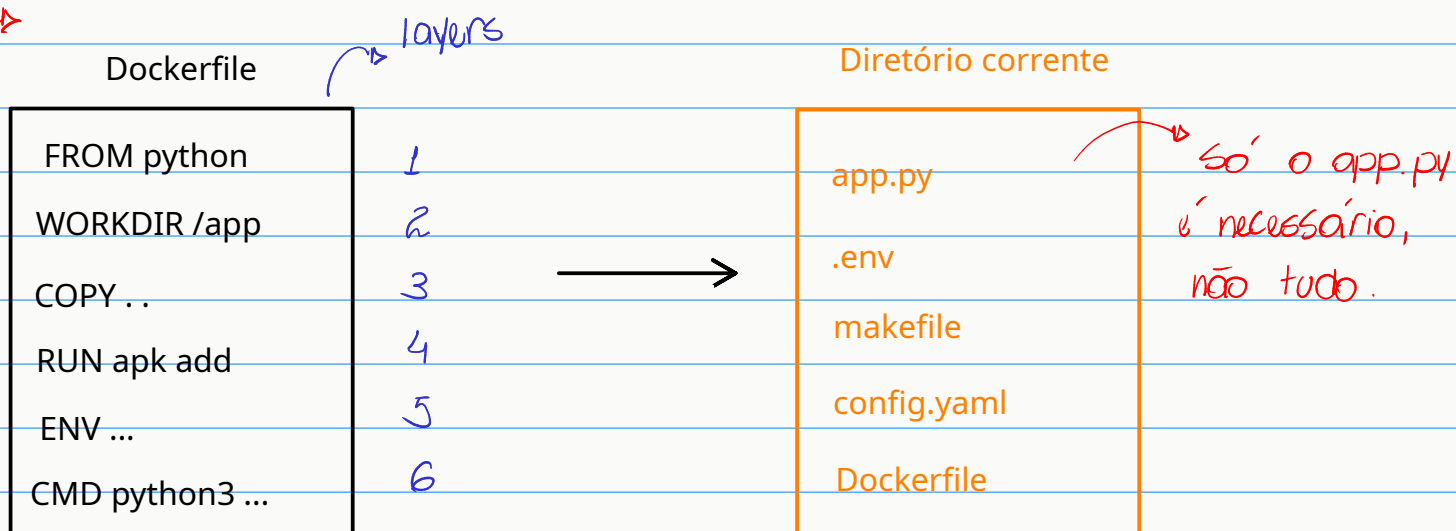
↳ Sistemas compartilhados (docker swarm)



Build de Imagens



Cache do Docker

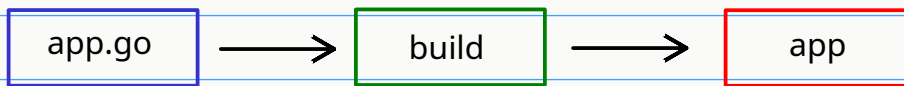


ordem importa

cuidado com "wildcards" → copiar só o necessário

Multi-staging build

↳ Geralmente utilizado para linguagens compiladas para otimizar tamanho/armazenamento.



↳ Ao invés de usar uma imagem para tudo, o processo é separado em **build** e **runtime**.



Dockerfile

```
FROM Golang-debian ... as builder
WORKDIR /app-go
RUN go build ...
COPY app
```

```
FROM scratch...
COPY --from builder
CMD ["go", "app"]
```

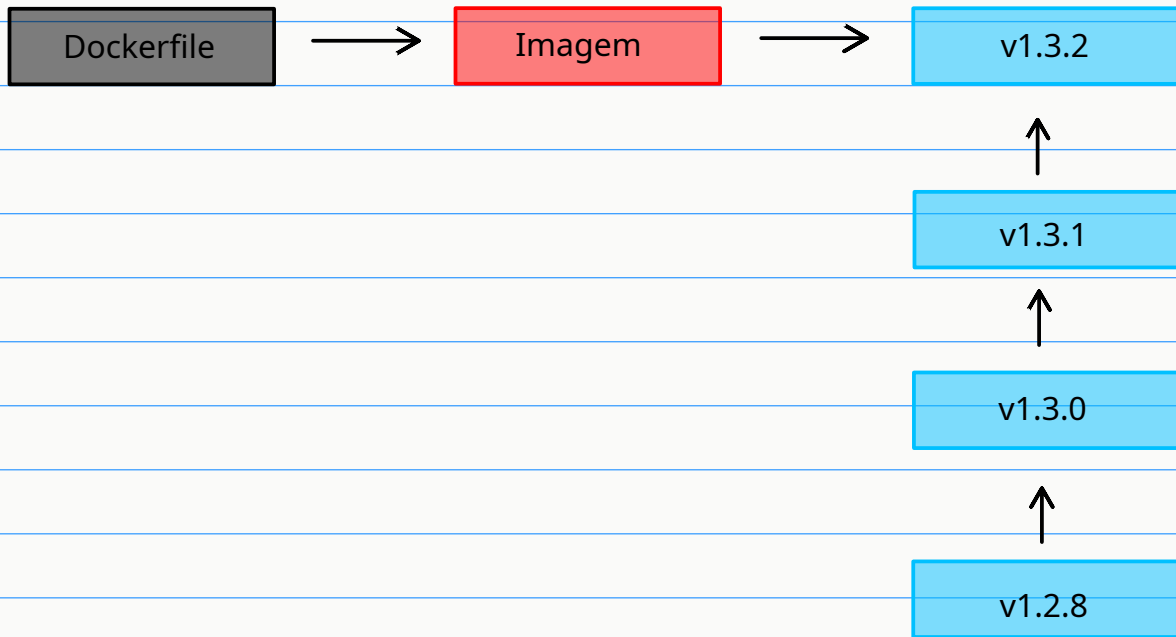
↳ 1GB
} auxiliar para realizar build e gerar binário

} imagem é construída em cima disso.

↳ 10MB

Utilização de tags

→ Garante o versionamento das imagens → geralmente o hash do commit é utilizado.



Commit → build → pagamento: v1.3.0 - hash commit
↳ <nome da imagem>: <tag>

Docker Compose

→ faz o provisionamento de tudo que é necessário para subir a aplicação.

↳ Containers, imagens, volumes, variáveis, rede, etc.

docker-compose.yml

- frontend
- backend
- postgresql

Docker swarm

↳ Orquestração de containers

Empresa
200 microserviços

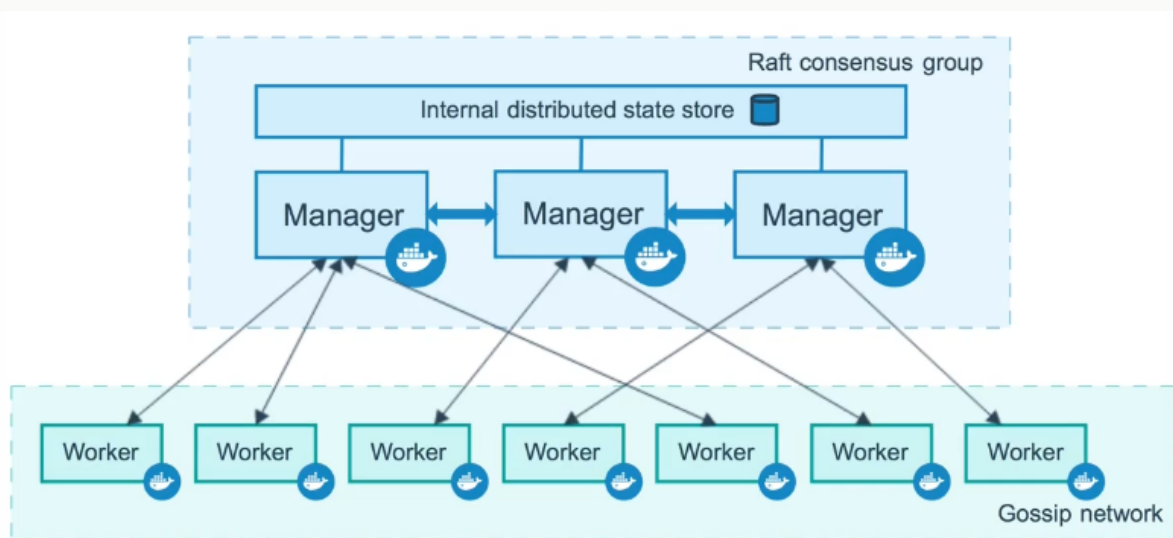
Escalar 4x

↳ docker swarm
800 containers

↳ Vantagens:

- 1 → Se um node morrer, workload é migrado
- 2 → Escala a quantidade de containers
- 3 → Gerenciamento centralizado
- 4 → Self-healing
- 5 → Load Balancing

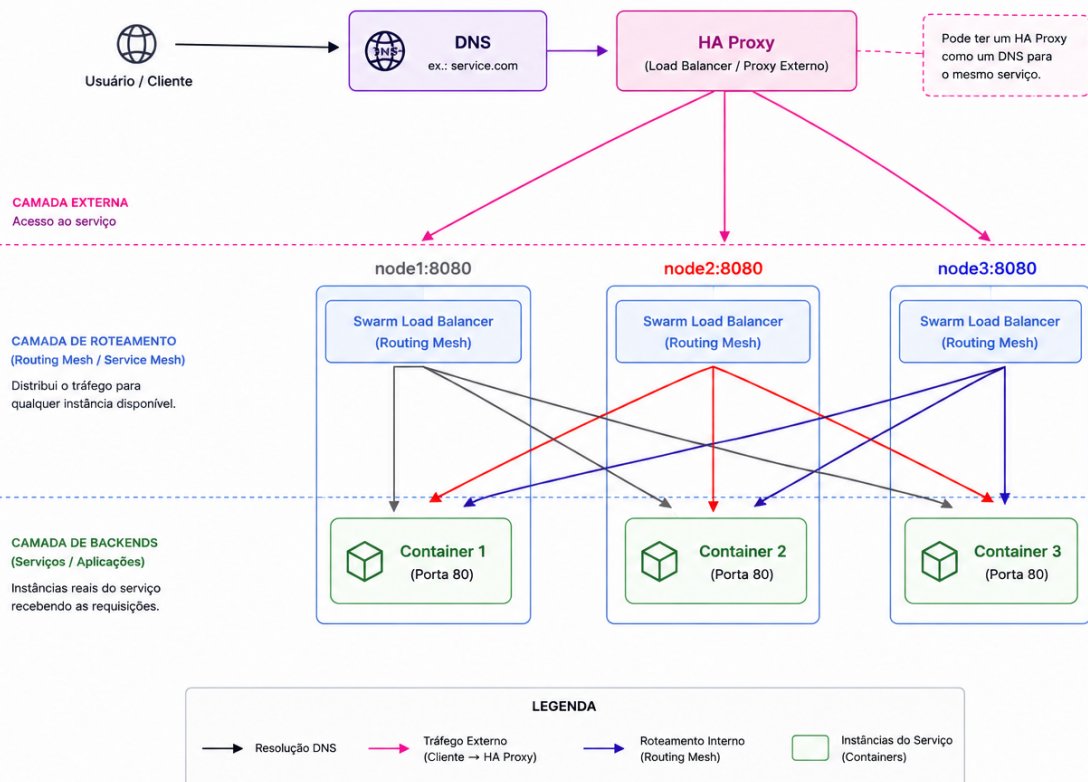
↳ Arquitetura docker swarm



Routing mesh

Habilita cada node receber requisições na portas publicadas.

↳ "-p 8080:80" => nginx service



Como funciona:

1. O cliente acessa o serviço via DNS (ou HA Proxy).
2. O HA Proxy recebe a requisição e encaminha para qualquer nó do Swarm (porta publicada ex.: 8080).
3. O Routing Mesh (Swarm Load Balancer) em cada nó distribui a requisição para qualquer instância disponível no cluster (pode ser local ou em outro nó).
4. A resposta retorna pelo mesmo caminho de volta até o cliente.

Observações:

- O HA Proxy fica fora do Swarm (camada externa).
- O Routing Mesh faz parte do Swarm (camada interna).
- As instâncias podem estar em qualquer nó do cluster.